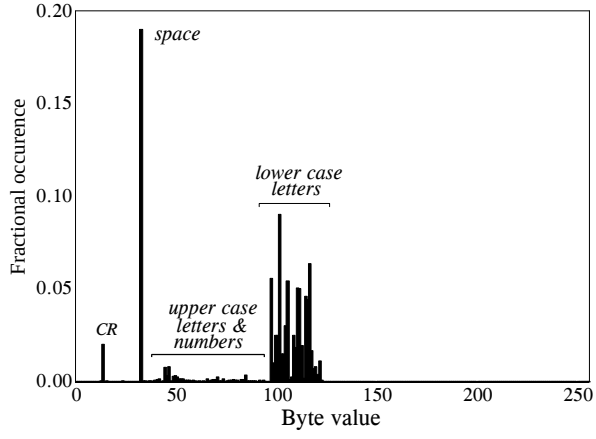

FIGURE 27-2
Histogram of text. This is a histogram of the ASCII values from a chapter in this book. The most common characters are the lower case letters, the space and the carriage return.

more bits. In this example, the *average* number of bits required per original character is: $0.96 \times 5 + 0.04 \times 13 = 5.32$. In other words, an overall compression ratio of: 8 bits/5.32 bits, or about 1.5:1.

Huffman encoding takes this idea to the extreme. Characters that occur most often, such the space and period, may be assigned as few as one or two bits. Infrequently used characters, such as: !, @, #, $ and %, may require a dozen or more bits. In mathematical terms, the optimal situation is reached when the number of bits used for each character is proportional to the logarithm of the character's probability of occurrence.

A clever feature of Huffman encoding is how the variable length codes can be packed together. Imagine receiving a serial data stream of ones and zeros. If each character is represented by eight bits, you can directly separate one character from the next by breaking off 8 bit chunks. Now consider a Huffman encoded data stream, where each character can have a variable number of bits. How do you separate one character from the next? The answer lies in the proper selection of the Huffman codes that enable the correct separation. An example will illustrate how this works.

Figure 27-3 shows a simplified Huffman encoding scheme. The characters *A* through *G* occur in the original data stream with the probabilities shown. Since the character *A* is the most common, we will represent it with a single bit, the code: 1. The next most common character, *B*, receives two bits, the code: 01. This continues to the least frequent character, *G*, being assigned six bits, 000011. As shown in this illustration, the variable length codes are resorted into eight bit groups, the standard for computer use.

When uncompression occurs, all the eight bit groups are placed end-to-end to form a long serial string of ones and zeros. Look closely at the encoding table of Fig. 27-3, and notice how each code consists of two parts: a number of zeros before a *one*, and an optional binary code after the *one*. This allows the binary data stream to be separated into codes without the need for delimiters or other marker between the codes. The uncompression program